

Guest lecture | Friday 9 October 2026, 9:30 AM | FX 313

Modern Full Stack Development

How a tap on your phone becomes a row in a database and comes back as a screen. We will build one, together, and put it on the internet.

K. Gift Brightson

Software Engineer, Cyrino India Pvt. Ltd., Chennai

For II CSE C

Coordinator: Mrs. M. Sathya, AP/CSE

Francis Xavier Engineering College
Vannarpettai, Tirunelveli

Five years of building things people use

I sat in a classroom like this one in 2019.

Front end

React, TypeScript, mobile apps.
The screens.

Back end

Java and Spring Boot, Node,
Python. The rules and the logic.

Database

PostgreSQL, SQL Server,
MongoDB. The memory that
survives.

Plus the things around them: Docker, cloud servers on AWS and Azure, automated deployments, and an AI search product built on a vector database.

Awarded **Engineer of the Year, 2024** at my previous company.

'17

B.E. Computer Science, 2017 to 2021

Same syllabus shape as yours. Same confusion in second year.

'21

Full stack developer

Admin dashboards, a mobile app, video calling built in.

'22

Full stack, then back end engineer

Hospital portals for patients and doctors. A data pipeline on Azure.

'24

Senior engineer, technical owner

Moved 20+ services of a global fitness company to the cloud with zero downtime. Built an AI assistant that answers from company documents.

'25

Software Engineer, Cyrino India, Chennai

A product-onboarding platform for a retail client abroad, and our own company's delivery platform.

The jobs behind the word "engineer"

Same degree. Four very different days. All of them start from today's three layers.

Full stack engineer

Owns one feature from the screen to the table. I built patient, admin and doctor portals this way, sitting next to the designer.

Back end engineer

Owns data and rules that many screens share. I built a pipeline that read vendor files every night, cleaned them, and loaded them into a database.

Senior engineer, technical owner

Owns the outcome, not just the code. Moving a payment page and a login service to a new cloud setup with no downtime meant more talking than typing.

Platform and AI engineer

Owns the architecture of something new. A pipeline that reads websites, breaks them into pieces, stores them so a chatbot can find the right piece in milliseconds.

Not on this list: "coder". Nobody is paid to type. People are paid to understand what the typing does.

What I build this year, with the names removed

A retail product-onboarding system

A retail chain abroad. Every new product they sell has to be entered, checked, audited and approved before it reaches shelves.

Front end React, TypeScript, data tables with thousands of rows

Back end Four small Node services: API, audit log, email, scheduler

Database SQL Server, with migration scripts for every change
Around it: Microsoft login, a message queue, automatic deploys from GitHub

Our own delivery platform

Projects, who works on what, hours, HR onboarding with e-signing, and a portal for candidates.

Front end Three React apps sharing one design

Back end One gateway and six NestJS services, one per subject

Database One PostgreSQL per service, Redis for speed
Around it: single sign-on, Docker, Terraform, a message queue

The names change every year. The shape never does: **front end** → **back end** → **database**.

You already have most of the pieces

From your Semester III and IV syllabus. I checked.

Object Oriented Programming using Java (24CS3602)

A class with fields is a table with columns. Today we attach a table to a class in one line.

Computer Organization and Architecture

RAM forgets on restart, disk remembers. That is the whole reason a database exists.

Operating Systems (next semester)

A server is a process listening on a port. Two programs, two ports, same laptop.

Data Structures (24CS3601)

JSON, the text that travels between layers, is just a HashMap and an ArrayList written out.

Database and SQL Programming (24CS4601, next semester)

You will see your first real SQL query today, six months early.

Soft skills (24PT3902)

Half of my week is explaining a bug to someone who is not an engineer. This counts.

Nothing today is outside your reach. It is your syllabus, connected.

Today, in five parts

1

One tap

What really happens when you order food on your phone.

2

Three layers

Each one explained with a little real code.

3

Build it

A todo app with sign up, in Java and React, built live in front of you.

4

Ship it

Put it on a real server with a real web address. How a name finds a computer.

5

Your next 3 years

What to learn when, what interviews test, and questions.

One rule for the next hour: when something on screen breaks, we do not hide it. We read the error together. That is the job.

What happens when you tap "Place order"?

You have done it a thousand times. Let's slow one second down.

One tap, six hops, about a third of a second

1 Your phone

The app collects your cart and sends a message over the internet: "new order, these items, this address".

2 A computer somewhere

The back end receives it. Are you logged in? Is the shop open? Is the price what the app claims?

3 The database

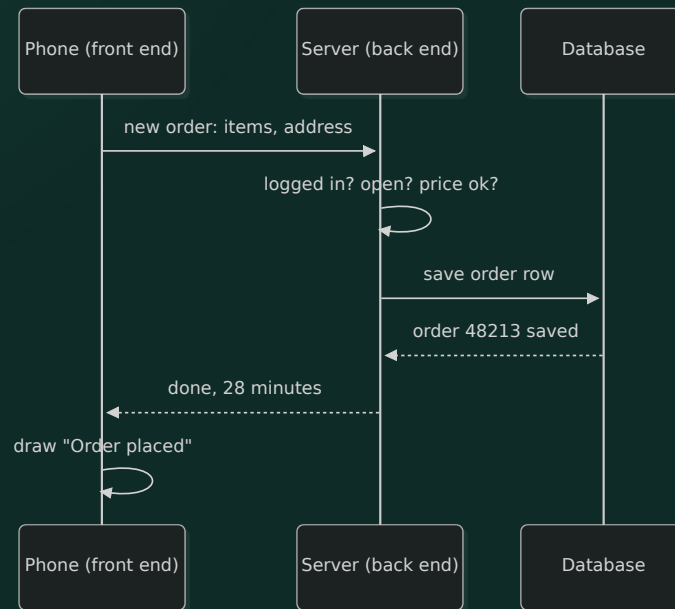
Saves the order as a row. Reduces stock. This survives even if the power goes.

4 The back end replies

"Done. Order 48213. 28 minutes."

5 Your phone

Reads the reply and draws the "Order placed" screen.



Full stack means you can write, fix and explain every one of those hops.

Not master all of them on day one. Follow a problem across them.

The three layers, as a halwa shop

Think of the famous evening shop in Tirunelveli with the long queue.

Front end

The counter. What the customer sees and touches. Runs on **your** phone or browser. HTML, CSS, JavaScript, React.

Back end

The kitchen. Takes the slip, applies the rules, makes the thing. Runs on a **server**. Java and Spring Boot.

Database

The store room. Ghee, sugar, wheat, kept safe and counted. Runs on a disk. PostgreSQL, MySQL, SQL Server.

The slip of paper is the API

The counter never walks into the kitchen. It writes a slip: "half kg halwa, counter 2". The kitchen never argues about the format. It reads the slip and cooks.

On the web the slip is called a **request**, the writing on it is **JSON**, and the fixed list of slips the kitchen accepts is the **API**.

Each layer only has to agree on the slip. That is why a React person and a Java person can work on the same app without reading each other's code.

Front end: the browser understands only three things

HTML says what exists

```
<h1>Canteen</h1>
<ul id="menu"></ul>
<button id="order">
  Order
</button>
```

CSS says how it looks

```
#order {
  background: #1f6f5f;
  color: white;
  border-radius: 8px;
  padding: 8px 16px;
}
```

JavaScript says what happens

```
const res = await fetch('/api/menu')
const items = await res.json()
menu.innerHTML = items
  .map(i => `<li>${i.name}</li>`)
  .join('')
```

Why React, then? Writing `innerHTML` by hand for fifty screens becomes a mess. React lets you say "for this data, the screen should look like this", and it redraws the page whenever the data changes. Angular, Vue and Svelte are the same idea with different spelling.

Back end: what a server actually does all day

Listen

Sit on a port, say 8080, and wait for requests. A port is a room number inside a computer.

Check

Who is asking? Are they allowed? Does the input make sense?

Decide

The rules. Is the shop open, is it sold out, does this discount apply.

Store

Read or write the database. The only layer allowed to touch it.

Reply

Send the answer as JSON with a status number: 200 ok, 201 created, 400 your fault, 404 not found, 500 my fault.

Why not do all this in the browser?

The browser belongs to the user. Anyone can press F12 and set `price = 0`. The back end is the part you control. So the rules live there.

Why Java and Spring Boot?

Most banks, telecoms and large shops in India run on it. Java catches many mistakes before the program runs. Spring Boot gives you a working server with one annotation.

Back end: a Spring Boot server in one file

```
@SpringBootApplication
public class CanteenApp {
    public static void main(String[] args) {
        SpringApplication.run(CanteenApp.class, args);
    }
}

@RestController
@RequestMapping("/api/menu")
class MenuController {

    private final MenuRepository repo;
    MenuController(MenuRepository repo) {
        this.repo = repo;
    }

    @GetMapping
    List<MenuItem> all() {
        return repo.findAll();
    }

    @PostMapping
    MenuItem add(@RequestBody MenuItem item) {
        return repo.save(item);
    }
}
```

One annotation, one server. Run `main` and a web server starts on port 8080.

This class answers requests whose address starts with `/api/menu`, and replies in JSON.

You ask for a repository, Spring hands you one. No `new`. This is called dependency injection. You will see it everywhere.

GET means "give me". A Java list goes out as a JSON array.

POST means "here is a new one". JSON comes in, becomes a Java object, is saved, goes back out with its new id.

Database: why not just a file?

A table is a shared, typed, searchable spreadsheet

id	name	price	available
1	Veg biryani	60	true
2	Masala dosa	40	true
3	Filter coffee	15	false

```
SELECT name, price
FROM menu_item
WHERE available = true
ORDER BY price;
```

That is SQL. You will meet it properly next semester in Database and SQL Programming.

- **Many people at once.** Five hundred students ordering at 12:30. A file would get corrupted. A database queues and locks correctly.
- **Nothing half done.** Power cut mid-save? A transaction either fully happens or not at all.
- **Fast at scale.** An index finds one row out of ten million in milliseconds. Reading a file line by line does not.
- **Relationships.** An order belongs to a student and has many items. SQL joins say this directly.
- **One language since 1974.** SQL has outlived every framework. Learn it properly once.

Relational databases (PostgreSQL, MySQL, SQL Server) for most things. MongoDB when the shape of the data keeps changing. Both are "the database layer".

How the layers talk: a request is just text

What the phone sends

```
POST /api/orders HTTP/1.1
Host: canteen.fxec.edu
Content-Type: application/json
Authorization: Bearer eyJhbGc...

{ "studentId": 2103, "items": [1, 2] }
```

What the server sends back

```
HTTP/1.1 201 Created
Content-Type: application/json

{ "id": 48213, "total": 100, "eta": "10 min" }
```

JSON is Data Structures, written down

The curly braces are a **HashMap**: key to value. The square brackets are an **ArrayList**. Nest them and you can describe anything.

The menu of allowed slips

Verb	Address	Means
GET	/api/menu	give me the list
GET	/api/menu/2	give me item 2
POST	/api/menu	here is a new one
DELETE	/api/menu/2	remove item 2

Agree on this table first, and the front end person and the back end person can work the same afternoon without waiting for each other. In companies we write it down and call it the API contract.

Let's build one

A todo app with sign up and log in. Java and Spring Boot behind, React in front, a real database under it.

You will see

A sign-up form, a log-in form, a list you can add to, tick, and delete.

You will watch me write

Two tables, five endpoints, three React files. About 150 lines, live, mistakes included.

You will take home

The whole project on GitHub, with the steps written out, to rebuild this weekend.

Before any code: install the tools once

Java and Node are the two engines. One tool installs both and keeps the versions right. Do this at home before the project, not during.

```
# Windows (PowerShell)
winget install jdx.mise

# Mac or Linux
curl https://mise.run | sh

# then, inside the project folder
$ mise install
java    temurin-17    installed
node    24.18.0          installed
pnpm     10.34.4         installed

$ java -version
$ node -v
```

The project has a tiny file called `mise.toml` that lists the versions. Everyone gets the same ones. That ends "works on my laptop".

mise

One tool for Java, Node, Python, anything. Works on Windows, Mac and Linux. This is what we use at work.

SDKMAN

Java-only, Mac and Linux only. Also good: `sdk install java 17-tem`. Pick one, not both.

Not this

Downloading an installer from a random site and clicking Next ten times. You will not remember what version you have, and neither will your teammate.

Step 1: a Spring Boot project in sixty seconds

Go to start.spring.io and choose

Setting	Pick
Project	Maven
Language	Java
Spring Boot	the latest stable (4.0.x)
Group / Artifact	<code>in.fxec</code> / <code>todo</code>
Java	17
Dependencies	Spring Web, Spring Data JPA, Validation, H2 Database

H2 is a database that lives inside the app while you learn, so there is nothing else to install. We swap it for PostgreSQL when we deploy.

Download, unzip, run

```
$ cd todo
$ ./mvnw spring-boot:run

.  ____          _ _   ____
/\  / ____'_ _ _ _ _  (  )_ _ _ _ _ \\\ \
...
Tomcat started on port 8080
Started TodoApplication in 2.1 seconds
```

Stop here and look. Your laptop is now a server. Open `localhost:8080` in the browser. The error page you see is Spring saying "I am alive, but you have not told me what to answer yet."

Step 2: the users table

A class from your OOP lab, with a table attached. Note what we do not store.

```
@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue
    private Long id;

    @Column(unique = true, nullable = false)
    private String email;
    private String name;

    // never the real password, only a scrambled version
    private String passwordHash;

    // handed out after login, sent back with every request
    private String token;
}
```

```
public interface UserRepository
    extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
    Optional<User> findByToken(String token);
}
```

Modern Full Stack Development, II CSE C

The table is called `users` because `user` is a reserved word in many databases. You learn these the hard way, once.

Unique email. The database refuses a second row with the same email. Rules you can push into the database are rules nobody can forget.

We never store the password. We store a hash: a one-way scramble. If the database leaks, the passwords do not. Any site that can email you your old password is doing it wrong.

The token is a long random string. It is how the server remembers you are logged in without asking for the password every time.

Step 3: sign up and log in

```
@Service
public class AuthService {
    private final UserRepository users;
    private final BCryptPasswordEncoder hasher
        = new BCryptPasswordEncoder();

    public User signup(String name, String email, String pw) {
        if (users.findByEmail(email).isPresent())
            throw new IllegalStateException("Already registered");
        var user = new User(name, email, hasher.encode(pw));
        user.setToken(UUID.randomUUID().toString());
        return users.save(user);
    }

    public User login(String email, String pw) {
        var user = users.findByEmail(email).orElseThrow(
            () -> new IllegalStateException("Wrong email or password")
        );
        if (!hasher.matches(pw, user.getPasswordHash()))
            throw new IllegalStateException("Wrong email or password");
        user.setToken(UUID.randomUUID().toString());
        return users.save(user);
    }
}
```

Sign up: refuse a duplicate, scramble the password, make a token, save. Four lines of rules. They live here, on the server, where nobody can edit them.

Log in: find the user, check the password against the scramble, hand out a fresh token.

Same message for "no such email" and "wrong password". Otherwise a stranger can find out who has an account.

Then a small controller maps `POST /api/auth/signup` and `POST /api/auth/login` to these two methods and returns `{ id, name, email, token }`. No password, no hash, ever leaves the server.

Step 4: the todos, and only yours

```
@Entity
public class Todo {
    @Id @GeneratedValue
    private Long id;
    private String title;
    private boolean done = false;
    @ManyToOne(optional = false)
    private User owner;
}

@GetMapping
List<TodoResponse> mine(
    @RequestHeader("Authorization") String bearer) {
    var me = auth.currentUser(bearer);
    return todos.findByOwnerOrderByIdDesc(me)
        .stream().map(TodoResponse::from).toList();
}

@PostMapping
@ResponseStatus(HttpStatus.CREATED)
TodoResponse add(
    @RequestHeader("Authorization") String bearer,
    @Valid @RequestBody NewTodo body) {
    var me = auth.currentUser(bearer);
```

Modern Full Stack Development

Many todos, one owner. `@ManyToOne` becomes a foreign key column in the table. Your DBMS course will call this a relationship.

Every request carries the token. The first thing we do is turn it into "who is this". No token, no list. The reply is 401, "please log in first".

Adding checks the title is not blank (`@Valid`) and attaches the todo to *you*. You cannot add to someone else's list even if you try.

Try it before any screen exists:

```
curl localhost:8080/api/todos -H "Authorization: Beare
```

If the API works in curl, every front end bug is a front end bug.

That halves your search.

Step 5: the React side, three files

```
$ pnpm create vite frontend --template react
$ cd frontend && pnpm install && pnpm dev
VITE ready in 300 ms
Local: http://localhost:5173/
```

```
// api.js: every call to the back end goes through here
export async function api(path, opts = {}) {
  const { method = 'GET', body, token } = opts
  const headers = { 'Content-Type': 'application/json' }
  if (token) headers.Authorization = 'Bearer ' + token
  const res = await fetch('/api' + path, {
    method, headers,
    body: body ? JSON.stringify(body) : undefined,
  })
  const data = res.status === 204 ? null : await res.json()
  if (!res.ok) throw new Error(data.error || 'Something went wrong')
  return data
}
```

`App.jsx` asks one question: do we know who you are? No: show the sign-up form. Yes: show the list.

`AuthForm.jsx` is a form with name, email, password. On submit it calls `api('/auth/signup', ...)` and hands the user up to `App`.

`TodoList.jsx` loads `/todos` when it appears, and adds, ticks or deletes by calling the API and updating its memory.

Two programs on one laptop: Spring on room 8080, Vite on room 5173. A three-line setting in `vite.config.js` forwards anything starting with `/api` to 8080, so the browser only ever talks to one address.

Run it, and watch the slip travel

Two terminals

```
# terminal 1, inside backend/  
$ ./mvnw spring-boot:run  
  
# terminal 2, inside frontend/  
$ pnpm dev
```

Then press F12, open Network, and sign up

POST /api/auth/signup status 201, reply has your token

GET /api/todos status 200, reply is []

POST /api/todos status 201, reply is your new todo

In terminal 1, Spring prints the SQL it ran

```
insert into users (email,name,password_hash,token,id)  
  values (?,?,,?,?)  
select t.id,t.done,t.owner_id,t.title from todo t  
  where t.owner_id=? order by t.id desc  
insert into todo (done,owner_id,title,id)  
  values (?,?,,?)
```

That is the whole lecture on one screen. A tap became a request, a request became a row, a row became a reply, a reply became pixels.

Open `localhost:8080/h2` and you can see the rows yourself.

The three things that break first, and what they mean

"blocked by CORS policy"

The browser refuses to let a page on 5173 call a server on 8080. It is protecting the user.

Fix: the Vite proxy from Step 5, or a small CORS setting in Spring. In production both sit behind one address and the problem disappears.

400 or 415 from Spring

You forgot `Content-Type: application/json`, or the JSON keys do not match the Java field names.

Fix: read the response body. Spring names the exact field that failed.

"Table users not found"

You renamed a field and the database did not follow.

Fix: while learning, `ddl-auto=update`. In real projects, migration files (Flyway) so a table never changes by surprise.

Nine times out of ten, the first line of the error is the answer. The engineers I trust most are not the ones who never see errors. They are the ones who read them calmly, all the way to the end, before touching anything.

Now put it on the internet

A real address anyone in this room can open on their phone.

What a server is, and what it costs

A server is a normal computer that never sleeps, in a building with good electricity and a fat internet cable. You rent it by the month. Rent a million of them with more buttons and people call it "the cloud".

Where

A data centre in Germany or Finland, for the one I use

Size

2 CPUs, 4 GB RAM. Less than your laptop. Enough for this class.

Cost

7 dollars a month, about 600 rupees. Two biryanis a week. Blame the AI boom for the price.

Access

A terminal over SSH. No screen, no mouse. Just you and a prompt.

```
$ ssh root@65.108.xx.xx
Welcome to Ubuntu 24.04 LTS

root@todo:~$ docker --version
Docker version 28.x

root@todo:~$ git clone <our repo> todo
root@todo:~$ cd todo/demo/deploy
root@todo:~$ cp .env.example .env # fill in the domain
root@todo:~$ docker compose up -d --build
[+] Running 3/3
 ✓ db          Started
 ✓ backend     Started
 ✓ web         Started
```

Docker packs each layer with everything it needs, so the server never asks "which Java?". The `compose.yml` file says: one PostgreSQL, one Spring Boot, one web server. One command starts all three.

How does a name find a computer?

You type `todo.giftson.org`. The internet only knows numbers. DNS is the internet's contacts app.

1 Your phone asks its contact book

"Do I already know the number for `todo.giftson.org`?" Not the first time.

2 It asks your network's helper

Jio, Airtel or the college Wi-Fi runs a "resolver": a librarian who finds things for you.

3 The librarian asks the root

"Who handles names ending in `.org`?" Thirteen groups of computers worldwide answer this all day.

4 Then the `.org` registry

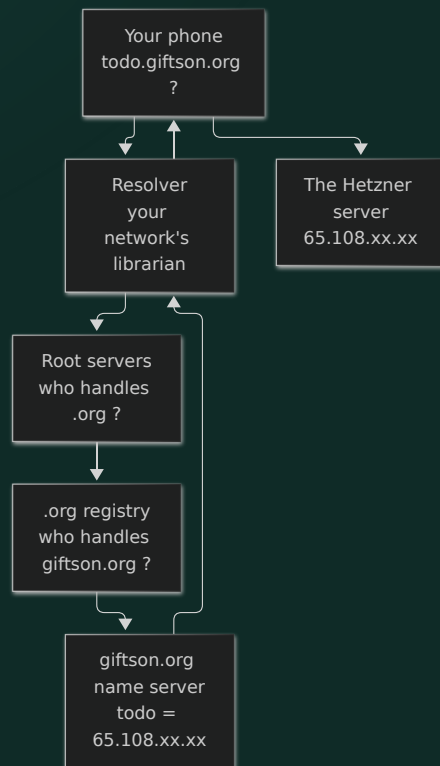
"Who handles `giftson.org`?" It replies with the address of my name server.

5 Then my name server

"`todo.giftson.org` is `65.108.xx.xx`." That line is an **A record**, typed into a form at my domain company.

6 Your phone connects to the number

And remembers it for a while (the TTL). After the first person, everyone in this room gets the answer instantly.



The last mile: address, room number, and the padlock

IP address = the building

`65.108.xx.xx` is the server. DNS gave us this.

Port = the room

Spring sits in room 8080.
PostgreSQL in 5432. The browser knocks on 443, the official front door for HTTPS.

HTTPS = the padlock

Everything between the phone and the server is scrambled, so the college Wi-Fi cannot read your password. A certificate proves the server is really `todo.giftson.org`.

```
# Caddyfile: the whole web server configuration
todo.giftson.org
```

```
handle /api/* {
    reverse_proxy backend:8080
}
handle {
    try_files {path} /index.html
    file_server
}
```

Caddy is the receptionist at the front door. It gets the padlock certificate by itself, for free, in about ten seconds. Requests starting with `/api` go to the kitchen on 8080. Everything else is the React app's files. Ten years ago this slide was a two-hour fight with configuration files. Now it is nine lines. The ideas did not change. The tools got kinder.

A plan by semester, not by hype

This year: foundations that never expire

- **Java, deeply.** Your OOP course is the perfect excuse. Classes, collections, exceptions, streams.
- **Data Structures, by hand.** Lists, hash maps, recursion. Two problems a week beats twenty in one night.
- **Git, every day.** Branch, commit, push. Even for lab records.
- **Three plain HTML pages** before touching React.
- **Next semester: SQL, properly.** Joins and indexes. Install PostgreSQL on day one of the course.

Third year: build whole things

- **Spring Boot + React + PostgreSQL,** one real project with login, a list, and a deploy. Today's app is the seed.
- **Put it on the internet.** A URL beats a zip file in every interview.
- **Docker,** one Dockerfile, one compose file.
- **Read other people's code,** one open-source project, one hour a week.
- **An internship.** Even six weeks, even small. The first one is the hardest to get.

Final year: depth and proof

- **Pick a side to go deeper:** back end, front end, data, cloud. Full stack is the base, not the ceiling.
- **Final-year project as portfolio.** Solve a real problem for a real person: your canteen, your department, a shop in Vannarpetai.
- **Interview practice:** DSA, fundamentals, and explaining your own project without notes.
- **Write it down.** README, a blog post, a LinkedIn post. Writing is how seniors are spotted.

Not on this list: twelve frameworks, forty certificates, every new AI tool. Breadth is cheap. Depth gets you hired.

What a fresher interview actually tests

1 Can you think in steps?

The DSA round. Not trick questions. Can you turn a problem into steps and code the steps cleanly. Arrays, strings and hash maps cover most of it.

2 Do you understand what you built?

"Walk me through your project." Then: "two users add a todo at the same time, what happens?" These questions live in the three layers.

3 Fundamentals

What is a primary key. How is a password stored. What does 404 mean. Today's deck is the checklist.

4 Will we enjoy working with you?

"I don't know, but here is how I would find out" beats bluffing, every time.

Questions I ask when I interview

- The API returns 500 for one user only. Where do you look first?
- The page is slow. Is it the front end, the back end, or the query? How would you know?
- Why can't we let the browser calculate the bill?
- Show me a Git commit you are proud of. Why that one?

None of these need a famous college or a 9.5 CGPA. They need one project you built end to end and understood end to end. Start it this semester.

Things I believed in second year

~~"I need to know everything before I start."~~

You learn by starting. I learned Terraform because a migration needed it that month, not before.

~~"Front end is easy, back end is for the smart ones."~~

Both are hard at depth. A fast, accessible form on a slow phone is as hard as a slow query.

~~"Failing the build means I am bad at this."~~

The build fails for everyone, daily. The skill is what you do in the next two minutes: read, not panic.

~~"Companies only want the newest framework."~~

Companies want people who can learn the next one. Java, SQL and HTTP have paid salaries for twenty-five years.

~~"AI will write the code, so why learn?"~~

AI writes the code. You own the consequence. Someone must know that the generated query locks the table.

~~"I am from a small college, so..."~~

So was I. Nobody has asked about my college since the first job. They ask what I built.

Free things that are actually good

Front end

- **MDN**, developer.mozilla.org. The reference for HTML, CSS, JS.
- **react.dev**, the official tutorial. Do "Tic-tac-toe" and "Thinking in React".
- **javascript.info**, the modern JS book.

Back end and Java

- **start.spring.io**, what we used today.
- **spring.io/guides**: "Building a RESTful Web Service", "Accessing Data with JPA". Fifteen minutes each.
- **boot.dev**, a back-end course that makes you build, with a free tier to start.

Database, tools, and the rest

- **SQLBolt** for practice, **postgresqltutorial.com** for depth.
- **free-for.dev**: a list of every free tier for developers. Hosting, databases, domains, email.
- **learngitbranching.js.org**, Git as a game.
- **roadmap.sh/full-stack**, the map. Do not try to finish it.

Homework, if you want it: start.spring.io, add Spring Web, Spring Data JPA and H2, and make `GET /api/todos` return a JSON list. One hour. That is the first hop of the six. Today's full project is in the repo on the last slide.

Questions

Code, career, college, Chennai rent. Anything.

About the code?

Any of the six hops, any of the steps.

About the job?

Interviews, internships, first salary, work from home.

About you?

"I am stuck at X, what next?" My favourite kind.

Build one thing you understand all the way down.

Then break it, read the error, and fix it. Do that enough times and you are not a student who codes. You are an engineer.

K. Gift Brightson

Software Engineer, Cyrino India Pvt. Ltd., Chennai

giftson2310@gmail.com

Today's code: [github.com/<your-handle>/fxec-fullstack](https://github.com/giftson2310/fxec-fullstack)

With thanks to Mrs. M. Sathya, AP/CSE, and the Department of Computer Science and Engineering, Francis Xavier Engineering College.